Newton-Raphson method

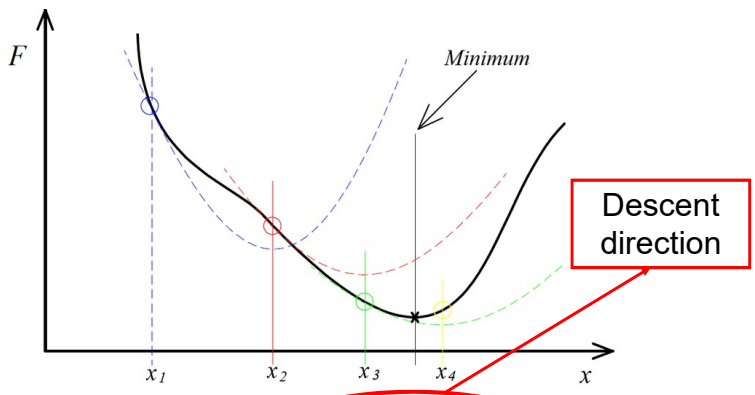
- The method takes inspiration from Newton-Raphson method to find the minimum of a nonlinear function $f: \mathbb{R}^{n\times 1} \longmapsto \mathbb{R}$
- If $f(\boldsymbol{x}) \in \mathcal{C}^2$ and can be approximated by a quadratic function in a neighborhood of $\boldsymbol{x}_k$, the method finds a sequence $\{\boldsymbol{x}_k\}_{k\in\mathbb{N}}$ converging to the minimum:

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \mathrm{H}_f^{-1}(\boldsymbol{x}_k) \cdot \nabla f(\boldsymbol{x}_k)$$

being $\mathrm{H}^{-1}(\boldsymbol{x}_k)$ ($\in \mathbb{R}^{n\times n}$) and $\nabla f(\boldsymbol{x}_k)$ ($\in \mathbb{R}^{n\times 1}$) the inverse of the Hessian matrix and the gradient of f calculated in $\boldsymbol{x}_k$.

37

Newton-Raphson method

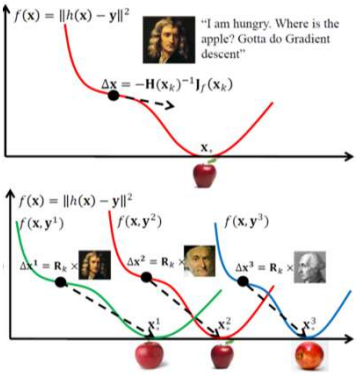


$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \mathrm{H}_f^{-1}(\boldsymbol{x}_k) \cdot \nabla f(\boldsymbol{x}_k)$$

38

Supervised Descent Method

- BUT:
  - SDM removes the constraints about smoothness of function f and properties of matrix H (required to be definite positive) by learning descent directions from data
  - This also improves computational complexity, because we do not have to compute $\mathrm{H}_f^{-1}(\boldsymbol{x}_k)$.



Pictures from (Xiong and De la Torre, 2013)

39

SDM: problem formulation

- Let us take an image $\boldsymbol{d} \in \mathbb{R}^{m\times 1}$(i.e., having *m* pixels)
- $\boldsymbol{x} \in \mathbb{R}^{2p\times 1}$ are the coordinates of *p* landmarks
- Faces in the training set are manually annotated and, for each of them, landmark coordinates are $\boldsymbol{x}_* \in \mathbb{R}^{2p\times 1}$
- The algorithm is initialized by assigning landmarks the initial coordinates $\boldsymbol{x}_0 \in \mathbb{R}^{2p\times 1}$
- $\boldsymbol{d}(\boldsymbol{x}) \in \mathbb{R}^{p\times 1}$ are the values of the landmark pixels
- $\boldsymbol{h}(\boldsymbol{x}) \in \mathbb{R}^{kp\times 1}$ is a vector of features describing the neighborhood of each landmark (*k* values per landmark)

40

SDM: problem formulation

- The problem of landmark localization can be formulated as finding the displacement $\Delta\boldsymbol{x}$ with respect to the initial coordinates $\boldsymbol{x}_0$ that minimizes the following function:

$$f(\boldsymbol{x}_0 + \Delta\boldsymbol{x}) = \|\boldsymbol{h}(\boldsymbol{x}_0 + \Delta\boldsymbol{x}) - \boldsymbol{h}(\boldsymbol{x}_*)\|^2$$

- That is, we are looking for:

$$\min_{\Delta\boldsymbol{x}} f(\boldsymbol{x}_0 + \Delta\boldsymbol{x}) = \min_{\Delta\boldsymbol{x}} \|\boldsymbol{h}(\boldsymbol{x}_0 + \Delta\boldsymbol{x}) - \boldsymbol{h}(\boldsymbol{x}_*)\|^2$$

41

SDM: problem formulation

- The Newton's method can find a sequence of displacements $\{\Delta\boldsymbol{x}_k\}_{k\in\mathbb{N}}$ converging to the minimum of f
- Such sequence can be proven to be:

$$\Delta\boldsymbol{x}_{k+1} = \Delta\boldsymbol{x}_k - 2\mathrm{H}_f^{-1}(\boldsymbol{x}_0 + \Delta\boldsymbol{x}_k)\, J_{\boldsymbol{h}}^T(\boldsymbol{x}_0 + \Delta\boldsymbol{x}_k)\big(\boldsymbol{h}(\boldsymbol{x}_0 + \Delta\boldsymbol{x}_k) - \boldsymbol{h}(\boldsymbol{x}_*)\big)$$

being:

- ${\mathrm{H}_f}^{-1}(\boldsymbol{x}_0 + \Delta\boldsymbol{x}_k) \in \mathbb{R}^{2p\times 2p}$ the inverse of the Hessian matrix of f calculated in $\boldsymbol{x}_0 + \Delta\boldsymbol{x}_k$.
- $J_{\boldsymbol{h}}^T(\boldsymbol{x}_0 + \Delta\boldsymbol{x}_k) \in \mathbb{R}^{2p\times kp}$ the transposed Jacobian matrix of $\boldsymbol{h}$ calculated in $\boldsymbol{x}_0 + \Delta\boldsymbol{x}_k$.

42

SDM: problem formulation

- This can be rewritten as:

$$\Delta \boldsymbol{x}_{k+1} = \Delta \boldsymbol{x}_k + \mathrm{R}_k \Delta \boldsymbol{\phi}_k$$

where:

- $\mathrm{R}_k = -2\mathrm{H}_f^{-1}(\boldsymbol{x}_0 + \Delta \boldsymbol{x}_k)\, J_{\boldsymbol{h}}^T(\boldsymbol{x}_0 + \Delta \boldsymbol{x}_k)$
- $\boldsymbol{\phi}_k = \boldsymbol{h}(\boldsymbol{x}_0 + \Delta \boldsymbol{x}_k)$, $\boldsymbol{\phi}_* = \boldsymbol{h}(\boldsymbol{x}_*)$
- $\Delta \boldsymbol{\phi}_k = \boldsymbol{\phi}_k - \boldsymbol{\phi}_* = \boldsymbol{h}(\boldsymbol{x}_0 + \Delta \boldsymbol{x}_k) - \boldsymbol{h}(\boldsymbol{x}_*)$

- In a more convenient way:

$$\Delta \boldsymbol{x}_{k+1} = \Delta \boldsymbol{x}_k + \mathrm{R}_k \boldsymbol{\phi}_k + \boldsymbol{b}_k$$

being $\boldsymbol{b}_k$ a bias term

- SDM learns matrices R_k and bias terms $\boldsymbol{b}_k$.

43

SDM: training

- Data for training consists of N faces $\boldsymbol{d}^i, i = 1 \dots N$ and the manually annotated positions of their landmarks $\boldsymbol{x}_*^i$
- For each face, by applying the above-mentioned equation, the first descent direction is $\Delta \boldsymbol{x}_1^i = \mathrm{R}_0 \boldsymbol{\phi}_0^i + \boldsymbol{b}_0$
- Thus the first training step consists of learning R_0 and $\boldsymbol{b}_0$
- Procedure in a nutshell:
  - Randomly assigning initial positions $\boldsymbol{x}_0^i$
  - Computing the new position $\boldsymbol{x}_1^i$, which is reached by following the first descent direction $\Delta \boldsymbol{x}_1^i$
  - Minimizing the error, i.e., the distance from the target $\boldsymbol{x}_*^i$

44

SDM: training

- More formally, being $\boldsymbol{X}_0^i$ a random vector expressing the randomness of the initial position $\boldsymbol{x}_0^i$:
  - $\Delta\boldsymbol{X}_1^i = \boldsymbol{X}_1^i - \boldsymbol{X}_0^i = \mathrm{R}_0\boldsymbol{\Phi}_0^i + \boldsymbol{b}_0$, being $\boldsymbol{\Phi}_0^i = \boldsymbol{h}(\boldsymbol{X}_0^i)$.
  - The new position which is reached is $\boldsymbol{X}_1^i = \boldsymbol{X}_0^i + \mathrm{R}_0\boldsymbol{\Phi}_0^i + \boldsymbol{b}_0$
  - The error $e(\boldsymbol{X}_0^i)$ we make is:

$$e(\boldsymbol{X}_0^i) = \|\boldsymbol{x}_*^i - \boldsymbol{X}_1^i\| = \|\boldsymbol{x}_*^i - (\boldsymbol{X}_0^i + \mathrm{R}_0\boldsymbol{\Phi}_0^i + \boldsymbol{b}_0)\|^2 =$$
$$= \|\boldsymbol{x}_*^i - \boldsymbol{X}_0^i - \mathrm{R}_0\boldsymbol{\Phi}_0^i - \boldsymbol{b}_0\|^2 = \|\Delta\boldsymbol{X}_{*0}^i - \mathrm{R}_0\boldsymbol{\Phi}_0^i - \boldsymbol{b}_0\|^2$$

being $\Delta\boldsymbol{X}_{*0}^i = \boldsymbol{x}_*^i - \boldsymbol{X}_0^i$.

45

SDM: training

- Assuming that random vector $\boldsymbol{X}_0^i$ has a probability density function $f_{\boldsymbol{X}_0^i}(\boldsymbol{x}_0^i)$, we then compute:

$$\underset{\mathrm{R}_0,\,\boldsymbol{b}_0}{\arg\min} \sum_{i=1}^{N} \mathbb{E}\left(e(\boldsymbol{X}_0^i)\right) = \underset{\mathrm{R}_0,\,\boldsymbol{b}_0}{\arg\min} \sum_{i=1}^{N} \int e(\boldsymbol{x}_0^i) f_{\boldsymbol{X}_0^i}(\boldsymbol{x}_0^i) d\boldsymbol{x}_0^i$$

- Supposing now that $f_{\boldsymbol{X}_0^i}(\boldsymbol{x}_0^i)$ is Gaussian, the integral is approximated using Montecarlo sampling, yielding:

$$\underset{\mathrm{R}_0,\,\mathbf{b}_0}{\arg\min} \sum_{i=1}^{N} \sum_{\boldsymbol{x}_0^i} \|\Delta\boldsymbol{x}_{*0}^i - \mathrm{R}_0\boldsymbol{\phi}_0^i - \boldsymbol{b}_0\|^2$$

This is the well-known linear least squares problem, which can be solved in closed-form

46

SDM: training and testing

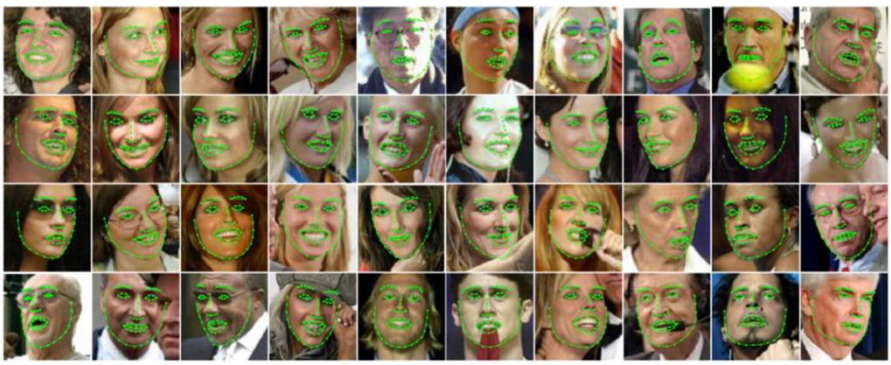
- Generalizing, R_{k+1} and $\boldsymbol{b}_{k+1}$ can be learnt by minimizing:

$$\underset{\mathrm{R}_{k+1},\, \mathbf{b}_{k+1}}{\arg\min} \sum_{i=1}^{N} \sum_{\boldsymbol{x}_{k+1}^{i}} \left\| \Delta \boldsymbol{x}_{*,k+1}^{i} - \mathrm{R}_{k+1} \boldsymbol{\phi}_{k+1}^{i} - \boldsymbol{b}_{k+1} \right\|^{2}$$

- The algorithm often converges in 4 or 5 steps
- During the testing phase, the algorithm starts from an initial position $\boldsymbol{x}_0$, and applies the sequence of displacements $\{\Delta \boldsymbol{x}_k\}_{k=1\ldots\mathrm{K}}$ (being K the number of steps needed to converge) to find the actual landmark positions.

47

SDM: examples

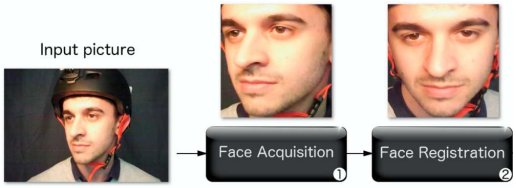


Example results on LFW dataset (http://vis-www.cs.umass.edu/lfw/)
(Figure from Xiong and De la Torre, 2013)

48

Face registration

- Misalignments produce large variations in the face appearance, resulting in large intra-class variance.
- Face registration refers to adjusting faces with respect to a common pre-defined reference coordinate system.



Pictures from (Allaert et al., 2018)

49

Classical approaches to FER

50

Feature Extraction

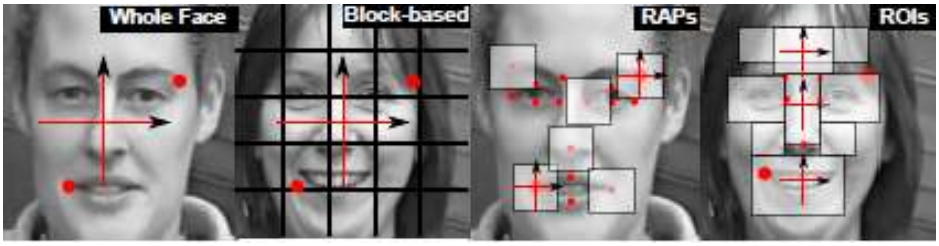
- Martinez et al. (2019) grouped feature extraction methods in 4 categories:
  - Appearance-based
  - Geometry-based
  - Motion-based
  - Hybrid methods
- Moreover features can be holistic or local according whether they are computed using a coordinate system relative to the entire face or relative to inner-facial structures

Appereance features

- Appearance features describe the color and texture of a facial region and can be divided into 5 major categories (Martinez et al., 2019):
  - Image intensity descriptors
  - Descriptors computed by using filter banks
  - Binarized local texture descriptors
  - Gradient-based descriptors
  - Two-layers descriptors

Appereance features

- Appearance features can be computed using different representation strategies:



Picture from (Martinez et al., 2019)

RAPs: Regions Around Points

ROI: Regions Of Interest

Image intensity descriptors

- Image intensity descriptors consist of the histogram or other statistical descriptors of the raw pixel intensities

- Advantages:
  - Easy to implement
  - Quick to compute.

- Limitations:
  - Very sensitive to all kinds of distractor variation.
  - Non-frontal head-poses are problematic.
  - They often need to be combined with other descriptors.

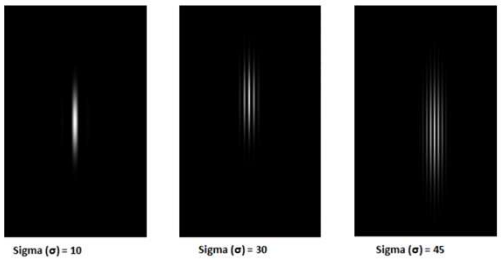
Filter banks

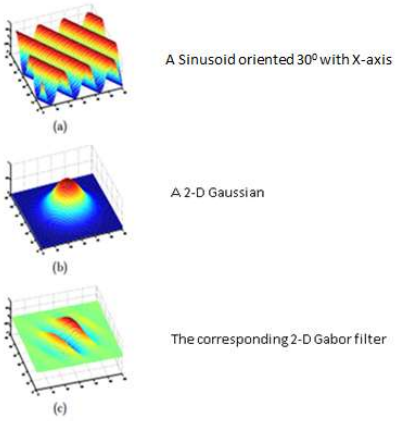
- Features are obtained by convolving a region of the input face with a set of filter bank
- Filter banks are used for analyzing deformations, i.e. shape and texture changes leading to high spatial gradients
- Gabor filters are commonly used in face analysis

Pictures from (Cruz, Bhanu, and Thankoor, 2013)

Gabor filter

- A Gabor filter is a special class of band pass filters. In the spatial domain it is a Gaussian kernel filter function modulated by a sinusoidal plane wave

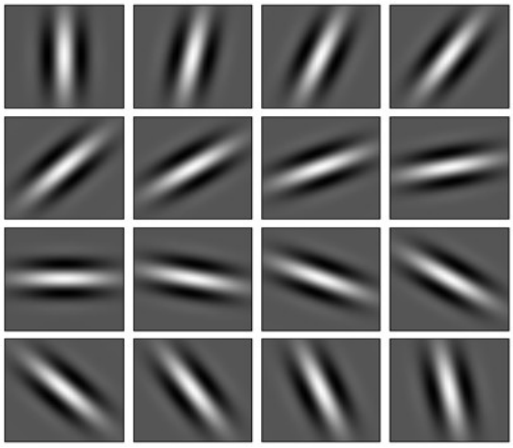




https://medium.com/@anuj_shah/through-the-eyes-of-gabor-filter-17d1fdb3ac97

Gabor filter banks

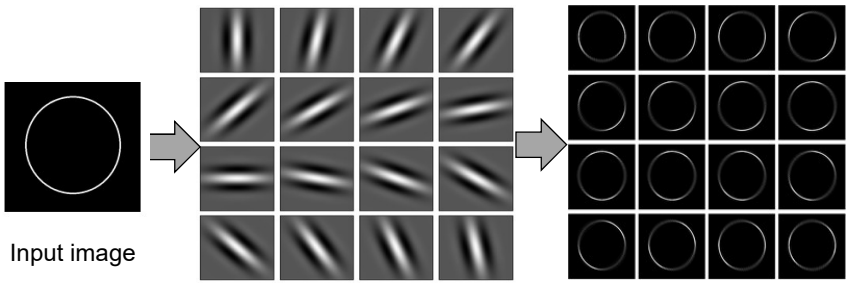
- The filter bank is made of Gabor filters with different orientations.



A bank of 16 Gabor filters oriented at angles of 11.250° apart (i.e. if the first filter is at 0°, then the second will be at 11.250°, the third will be at 22.50°, and so on)

https://medium.com/@anuj_shah/through-the-eyes-of-gabor-filter-17d1fdb3ac97

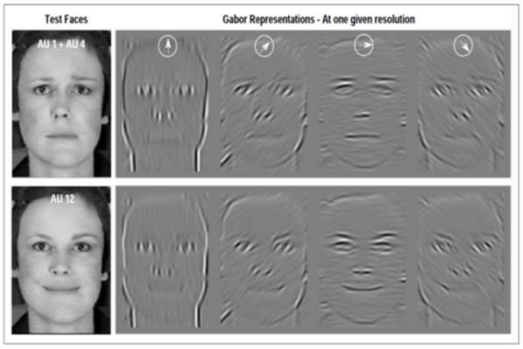
Gabor filter banks



https://medium.com/@anuj_shah/through-the-eyes-of-gabor-filter-17d1fdb3ac97

Gabor filter banks

- The magnitude values of the complex response of each filter for each orientation are concatenated into a feature vector.



Gabor representations of two facial expression obtained with a bank of 4 Gabor filters.
Picture from (Fasel and Luettin, 2003)

Gabor filter banks

- Advantages:
  - They can be a powerful representation, provided that the parametrization is correct
  - They are robust to small registration errors
- Limitations:
  - The resulting dimensionality is very large
  - Computational cost is high
  - Consequently, it is hard to make them work in real-time